

LEA-CTR 암호화와 LSH-HMAC 해시를 통한
MODBUS 프레임 종단간 암호화 프로그램
사용 매뉴얼

2026. 08.

목차

1. 개요	3
2. 소스코드 구성	5
3. 빌드 방법	7
4. 인터페이스	9
5. 데모 프로그램 사용법	26
6. 테스트 방법	27
7. 참조 문헌	31

1. 개요

이 프로젝트는 Modbus RTU 통신에 기밀성(암호화)과 무결성/인증(HMAC)을 더하여 종단간 암호화를 달성하는 데에 그 목적이 있다. 국산 경량 블록 암호인 LEA-CTR을 활용한 암호화와 국산 해시함수 HMAC-LSH256 인증을 사용하여 빠르고 안전한 암호화 및 복호화를 진행한다.

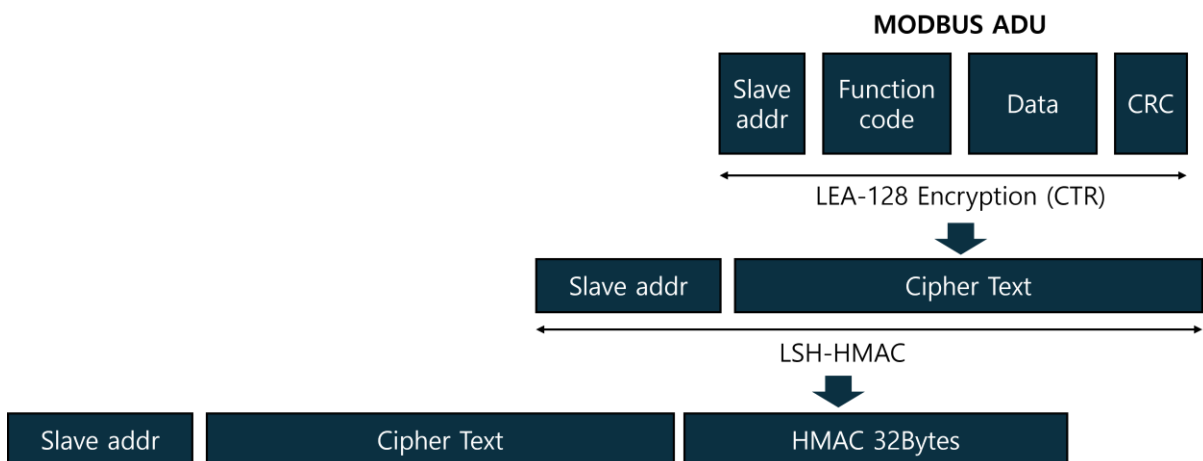
1.1 폴더 구조

docs/	매뉴얼 및 참고 문헌
crypto/	LEA, LSH/HMAC 원시 함수
keymgmt/	키 테이블 + 방향별(m2s/s2m/bc) CTR 카운터 상태
framing/	[addr ciphertext hmac] 와이어 프레임 구조
modbus/	PDU 내용물 (Function Code, 코일/레지스터 데이터 모델, 예외 응답)
demo/	위 계층들을 엮은 실행 가능한 데모

[표 1] 폴더 구조

폴더 구조는 [표 1]과 같이 기능 단위로 모듈화 하여 정리하였다.

1.2 프레임 구조



[그림 1] 프레임 구조

[addr(1)] [ciphertext = LEA-CTR(addr + PDU + CRC16)] [hmac(32)]

- addr: 평문으로 한 번 더 앞에 붙음 - 복호화 전에 프레임을 라우팅해야 하므로 필요하다.
- ciphertext: Modbus ADU(주소+PDU+CRC16) 전체를 LEA-CTR로 암호화한 암호문. CTR 카운터 자체는 전송되지 않고 송수신측 양쪽이 keymgmt/ctr_state.h로 각자 추적한다.
- hmac: [addr] || [ciphertext]에 대한 HMAC-LSH256 (32바이트).

1.3 설계 근거

왜 CTR 카운터를 전송하지 않는가?

키 관리 서버에서 안전하게 받아온다고 가정하고, 코드에서는 마스터와 슬레이브가 각자 독립적으로 카운터를 증가시키며 같은 순서로 프레임을 처리한다고 약속한다. 프레임 하나라도 유실되면 두 쪽의 카운터가 어긋남에 주의해야 한다.

왜 해시가 아닌 HMAC인가?

hmac_lsh256()은 암호 키가 들어간 구성으로, 평문 해시는 아무나 계산할 수 있으므로 "이 프레임이 변조되지 않았다"만 보장하고 "이 프레임을 우리가 공유한 키를 가진 쪽이 만들었다"는 보장하지 못하므로 보안 취약점 해소를 위해 HMAC을 사용한다.

왜 CRC16을 암호문 안에도 두는가?

secure_frame_verify_and_decrypt()가 HMAC 검증에 성공한 뒤 복호화하고 CRC를 다시 확인하는데, 이 CRC 검증은 카운터가 어긋났을 때의 1차 진단 신호로 확인한다.

HMAC은 키가 맞는 발신자가 보냈다는 것만 보장하고, 복호화에 쓴 카운터가 송신 측과 일치했는지는 보장하지 않으므로, CRC가 그 어긋남을 잡아내는 역할을 겸한다.

보안 상 제약사항

- **재전송(replay) 방지 미적용:** ctr_state_validate_incoming()이 재전송 탐지 로직을 구현하고 있지만, 코드 전체에서 실제로 호출되는 곳이 없다. 현재 수신측은 "다음에 올 프레임은 다음 카운터를 쓸 것"이라 가정하고 복호화 시도. 실제 환경에서는 재전송 시 탐지 로직이 필요하다.

- **HMAC 비교가 상수 시간이 아님:** `secure_frame.c`가 `memcmp()`로 HMAC을 비교해 시간 복잡도 $O(N)$.
- **카운터 손실 가능:** `ctr_state_persist()`는 매 호출마다 파일을 단순 재작성하는 방식으로, 기록 도중 전원이 끊기면 카운터 파일이 손상될 수 있어 실제 트래픽 처리 전에 재검토가 필요하다.
- **modbus/의 데이터 모델은 데모용:** 코일/레지스터 128개 고정 크기 배열이며 실제 슬레이브 장비의 레지스터 맵을 반영하지 않는다.

2. 소스코드 구성

2.1 crypto/ (LEA, LSH 원본 및 참조 함수)

파일명	내용
<code>lea.h/lea.c</code>	LEA 키 스케줄 생성 (<code>lea_key_schedule</code>)
<code>lea_ctr.h/lea_ctr.c</code>	LEA-CTR 암호/복호화 담당
<code>hmac_lsh.h/hmac_lsh.c</code>	HMAC-LSH256 담당
<code>lea_ref/</code>	LEA 라이브러리
<code>lsh_ref/</code>	LSH/HMAC 라이브러리

[표 2] crypto/ 하위 파일 목록

2.2 keymgmt/ — 키 테이블과 카운터 상태

파일명	내용
<code>key_store.h/key_store.c</code>	slave별/방향별 암호화·MAC 키 테이블, <code>keys.txt</code> 로더
<code>ctr_state.h/ctr_state.c</code>	slave별/방향별 CTR 카운터 상태 관리
<code>keys.txt</code>	테스트용 키 소스 파일

[표 3] keymgmt/ 하위 파일 목록

2.3 framing/ — 프레임 생성

파일명	내용
secure_frame.h/secure_frame.c	[addr ciphertext hmac] 프레임 조립/파싱, 암호화 +HMAC 결합(encrypt_and_build), 복호화+HMAC 검증 결합(verify_and_decrypt)

[표 4] framing/ 하위 파일 목록

2.4 modbus/ — Modbus PDU 로직

파일명	내용
modbus_pdu.h/modbus_pdu.c	Function Code 8종의 요청/응답 PDU 조립, 데모용 코일/레지스터 데이터 모델, Modbus 예외 응답
modbus_pdu_selftest.h/ modbus_pdu_selftest.c	요청 PDU 기반 응답(예외 응답) PDU 정상 생성 여부 테스트

[표 5] modbus/ 하위 파일 목록

2.5 demo/ — 실행 가능한 데모와 공용 유틸리티

파일명	내용
main.c	프로토타입(secure_demo) - 마스터/슬레이브 모드, 키 초기화, 셀프 테스트, 환경 설정, 실행
secure_send_demo.c	프레임 송신 데모(secure_send_demo) - 1~123개의 레지스터로 프레임 크기를 달리하여 프레임 스트레스 테스트
secure_rcv_demo.c	프레임 수신 데모(secure_rcv_demo) - 실제 하드웨어로 왕복 검증
serial_port.h/serial_port.c	COM/tty 포트 열기/읽기/쓰기, Modbus T3.5 유희 간격 계산(modbus_t35_us)

key_paths.h/key_paths.c	실행 파일 상대 경로 계산, keys.txt 탐색
demo_log.h/demo_log.c	파일 로깅 유틸리티
rs485_probe.c	커널 RS-485 드라이버 지원 여부 확인용

[표 6] demo/ 하위 파일 목록

3. 빌드 방법

3.1 Makefile 대상

make	secure_send_demo, secure_rcv_demo, secure_demo 모두 빌드
make clean	빌드 산출물 정리
make NO_NEON=1	NEON 대신 일반 C 경로로 빌드

[표 7] Makefile 방법

3.2 아키텍처 자동 감지 (ARCH := \$(shell uname -m))

ARCH	LEA SIMD 백엔드	비고
aarch64	NEON (NO_NEON=1이면 일반 C)	실제 타겟(Raspberry Pi 5 / CM5)
armv7l	NEON (NO_NEON=1이면 일반 C)	
그 외(x86_64 등)	일반 C만 (SIMD 없음)	Windows 등에서 개발용 빌드

[표 8] 지원 아키텍처

3.3 make가 없는 환경(예: Windows)의 수동 빌드

```
gcc -O2 -Wall -DNO_AVX2 -DNO_XOP -DNO_PCLMUL -DNO_SSE2 \#
demo/main.c \#
```

```

framing/secure_frame.c keymgmt/key_store.c keymgmt/ctr_state.c ₩
crypto/lea.c crypto/lea_ctr.c crypto/hmac_lsh.c ₩
crypto/lsh_ref/src/lsh.c crypto/lsh_ref/src/lsh256.c crypto/lsh_ref/src/lsh512.c
crypto/lsh_ref/src/hmac.c ₩
crypto/lea_ref/lea_base.c crypto/lea_ref/lea_core.c crypto/lea_ref/lea_online.c ₩
crypto/lea_ref/lea_gcm_generic.c crypto/lea_ref/lea_t_fallback.c
crypto/lea_ref/lea_t_generic.c ₩
crypto/lea_ref/cpu_info_ia32.c ₩
demo/serial_port.c demo/key_paths.c modbus/modbus_pdu.c
modbus/modbus_pdu_selftest.c ₩
-o secure_demo.exe -lws2_32

```

[표 9] gcc 빌드 명령어

secure_send_demo/secure_rcv_demo를 만들 때는 demo/main.c
를 demo/secure_send_demo.c(또는 secure_rcv_demo.c)로 바꾸고 demo/demo_log.c를 추
가한다. secure_rcv_demo는 modbus/modbus_pdu.c도 필요하지만 secure_send_demo는
필요 없다(하단 [표 10] 참고).

3.4 실행 파일별 필요 오브젝트

세 실행 파일은 공통 오브젝트(\$(LIB_OBJS): framing/+keymgmt/+crypto/+demo
/serial_port.c+demo/key_paths.c)를 공유하고, demo_log와 modbus_pdu에 대한 필요 여
부는 다음 [표 10]과 같다.

실행 파일	\$(LIB_OBJS)	demo_log.o	modbus_pdu.o
secure_demo (main.c)	필요	불요 (print_hex 사용)	필요
secure_send_demo	필요	필요	불요 (슬레이브 역할 없음, 고 정 함수 0x10 스왑만 함)
secure_rcv_demo	필요	필요	필요

[표 10] 실행 파일별 필요 오브젝트 일람

4. 인터페이스

4.1 crypto/

LEA/LSH의 호출부. 이 세 함수는 모두 입력 값이 정해진 길이/형식만 맞으면 실패하지 않는 순수 계산 함수라 반환 값이 없고(void), 별도의 예외/경계 조건도 없다.

4.1.1 lea_key_schedule

void lea_key_schedule(const uint8_t key[16], lea_key_schedule_t *ks)

입력: key 16바이트 LEA-128 마스터키

출력: ks에 라운드 키 채운다.

```
uint8_t enc_key[16] = { 0x01, 0x02, /* ...암호 키... */};  
  
lea_key_schedule_t ks;  
  
lea_key_schedule(enc_key, &ks);  
  
// 이후 lea_ctr_crypt(&ks, ...)로 암호/복호화
```

[표 11] lea_key_schedule 사용 예시

4.1.2 hmac_lsh256

void hmac_lsh256(const uint8_t *key, size_t key_len, const uint8_t *msg, size_t msg_len, uint8_t out[32])

입력: key/key_len MAC 키, msg/msg_len 메시지.

출력: out에 32바이트(HMAC_LSH_FULL_SIZE) HMAC-LSH256 값을 담음.

```
uint8_t mac_key[16] = { /* ... */};  
  
uint8_t tag[HMAC_LSH_FULL_SIZE];  
  
hmac_lsh256(mac_key, KEY_SIZE, wire_frame, wire_frame_len, tag);
```

[표 12] hmac_lsh256 사용 예시

4.2 keymgmt/key_store.h

slave별 · 방향별(master to slave; m2s / slave to master; s2m / broadcast; bc) 암호화 키와 MAC 키를 저장하는 테이블을 관리한다(프로세스 전역, static 배열).

key_direction_t는 DIR_MASTER_TO_SLAVE, DIR_SLAVE_TO_MASTER, DIR_BROADCAST로 구성된다. LEA-128에서 KEY_SIZE 상수는 16이고, 슬레이브 주소 유효 범위는 0~247이다.

(KEY_STORE_MAX_ADDR, Modbus 주소 체계와 동일) — 이 범위를 넘는 slave_addr은 모든 함수에서 -1로 예외 처리된다.

4.2.1 key_direction_index

```
int key_direction_index(key_direction_t dir, int *out);
```

매개변수: dir [in] 방향 값, out [out] 0..2 테이블 인덱스.

설명: key_direction_t를 테이블 인덱스로 매핑한다. key_store.c와 ctr_state.c가 이 매핑을 공유(둘 다 (addr, dir) 테이블을 색인). 성공 시 0, dir이 세 값 중 하나가 아니면 -1 반환.

동작/경계 조건: 이 함수는 다른 모든 key_store_*/ctr_state_* 함수 내부에서 호출되므로, 잘못된 key_direction_t 값(예: 정수를 강제 캐스팅해 3 이상을 넘긴 경우)을 넘기면 그 상위 함수들도 전부 예외 반환한다(-1 또는 0).

4.2.2 key_store_lookup

```
int key_store_lookup(uint8_t slave_addr, key_direction_t dir, directional_keys_t *out);
```

매개변수: slave_addr [in] 0~247, dir [in], out [out] 암호화/MAC 키 쌍.

설명: 주어진 방향에서 slave_addr의 키 쌍을 조회한다. 성공 시 0, 등록되어 있지 않으면 -1 반환.

동작/경계 조건: 프로세스 시작 직후(아무것도 provision/load_file하지 않은 상태)에는 모든 (addr, dir) 조합이 미등록 상태라 항상 -1이 반환된다. framing/secure_frame.c는 이 -1을 SECURE_FRAME_ERR_NO_KEY로 변환해 호출자에게 전달한다.

```
directional_keys_t keys;

if (key_store_lookup(1, DIR_MASTER_TO_SLAVE, &keys) == 0) {
```

```

    /* keys.enc_key, keys.mac_key 사용 가능 */
} else {

    printf("slave 1의 m2s 키가 등록되어 있지 않음\n");

}

```

. [표 13] key_store_lookup 사용 예시

4.2.3 key_store_provision

int key_store_provision(uint8_t slave_addr, key_direction_t dir, const directional_keys_t *keys);

매개변수: slave_addr [in], dir [in], keys [in].

설명: 주어진 방향에서 slave_addr의 키 쌍을 등록(또는 교체). 성공 시 0, slave_addr이 247을 넘거나 dir이 잘못된 값이면 -1 반환.

동작/경계 조건: 이미 등록된 (addr, dir)에 다시 호출하면 키뿐만 아니라 initial_ctr도 무조건 0으로 리셋된다. key_store_load_file()로 파일에서 읽어온 initial_ctr이 있어도 그 뒤에 key_store_provision()을 호출하면 덮어써진다. 순서를 바꿔 부르면(먼저 provision, 나중에 load_file) 파일의 initial_ctr 값이 최종적으로 남는다.

```

directional_keys_t dk;

memcpy(dk.enc_key, my_16_byte_key, KEY_SIZE);

memcpy(dk.mac_key, my_16_byte_mac_key, KEY_SIZE);

key_store_provision(1, DIR_MASTER_TO_SLAVE, &dk); /* initial_ctr은 0으로 설정됨 */

```

[표 14] key_store_provision 사용 예시

4.2.4 key_store_load_file

int key_store_load_file(const char *path);

매개변수: path [in] 키 파일 경로. 기본 파일명은 호출자가 정함(이 프로젝트의 데모들은 "keys.txt"를 관례로 씀).

설명: "<addr> <dir:m2s|s2m|bc> <enc_key:16자> <mac_key:16자> <initial_ctr:16진수 8자

>" 형식의 파일을 한 줄씩 읽어 테이블에 등록. #로 시작하는 줄과 빈 줄은 건너뛴. 불러온 항목 수를 반환, 파일을 열 수 없으면 -1.

동작/경계 조건: 형식이 안 맞는 줄(필드 개수가 5개가 아님, enc_key/mac_key 길이가 정확히 16자가 아님, addr이 247 초과, dir 토큰이 m2s/s2m/bc가 아님)은 **에러 없이 건너뛴다** – 반환 값은 오직 "성공적으로 파싱해 등록한 키의 수"이므로, 파일에 오타가 있어도 프로그램은 그냥 그 줄을 무시하고 계속 진행한다. 파일 전체가 형식에 맞지 않으면 유효한 키가 없으므로 반환 값 0(demo_load_keys()는 0 이하를 실패로 취급).

```
int loaded = key_store_load_file("keys.txt");

if (loaded <= 0) {

    printf("키를 하나도 불러오지 못함\n");

}
```

[표 15] key_store_load_file 사용 예시

4.2.5 key_store_get_initial_ctr

int key_store_get_initial_ctr(uint8_t slave_addr, key_direction_t dir, uint32_t *out);

매개변수: slave_addr [in], dir [in], out [out] key_store_load_file()이 읽은 초기 CTR 값(16진수 8자를 uint32_t로 파싱한 값). 기본값: key_store_provision()만으로 등록된 항목은 항상 0(위 참고).

설명: ctr_state가 그 (addr, dir)의 카운터를 처음 필요로 할 때 시드 값으로 사용. 성공 시 0, 미등록이면 -1. 이 함수를 직접 호출할 일은 거의 없고,

ctr_state_next_outgoing()/ctr_state_validate_incoming()이 내부적으로 호출한다.

4.3 keymgmt/ctr_state.h

slave별·방향별 CTR 카운터 상태 테이블 관리(다음 송신 시 사용할 카운터, 지금까지 수신된 카운터 최댓값). 이 테이블도 key_store처럼 프로세스 전역이며 기본값은 미 설정(첫 호출 시 key_store_get_initial_ctr()로 초기화).

4.3.1 ctr_state_next_outgoing

uint32_t ctr_state_next_outgoing(uint8_t slave_addr, key_direction_t dir, size_t msg_len);

매개변수: slave_addr [in], dir [in], msg_len [in] 이 카운터로 암호화될 평문 ADU 바이트 길이

설명: 아직 사용되지 않은 다음 송신 카운터 값을 반환하고 저장된 카운터를 증가시킨다. LEA-CTR은 16바이트 블록마다 카운터를 1씩 증가시키므로 $\text{ceil}(\text{msg_len}/16)$ 개의 연속 값이 이번 메시지에서 소비된다(msg_len == 0이어도 최소 1블록은 예약됨).

동작/경계 조건: 새 메시지마다 정확히 한 번만 호출해야 한다. 같은 (addr, dir)에 대해 의도치 않게 두 번 연달아 호출하면(예: 암호화 함수 안에서 한 번 소비된 뒤, 그 결과를 복호화 하려고 또 호출) 두 번째 호출은 다음 메시지용 카운터를 돌려주므로 복호화가 실패한다. secure_frame.c는 encrypt와 decrypt 경로에서 각각 정확히 한 번씩만 이 함수를 호출하도록 설계되어 있다. slave_addr이 247을 넘거나 dir이 잘못된 값이면 0을 반환(에러 코드가 아니라 유효한 첫 카운터 값과 구분이 안 되므로 호출 전에 입력을 스스로 검증해야 함).

4.3.2 ctr_state_validate_incoming

int ctr_state_validate_incoming(uint8_t slave_addr, key_direction_t dir, uint32_t ctr);

매개변수: slave_addr [in], dir [in], ctr [in] 수신된 카운터 값.

설명: ctr < highest_accepted면 거부(재전송 공격), ctr == highest_accepted면 허용(정상 재시도 — 호출자는 새 메시지로 취급하지 말고 재처리해야 함), ctr > highest_accepted면 허용 후 상한 갱신. 처리해야 하면 1, 거부해야 하면 0 반환.

동작/경계 조건: 이 함수는 코드 전체에서 선언·구현만 되어 있고 아직 실제로 호출되는 곳이 없다. 재전송 방지가 필요해지면 API를 사용하면 된다. 지금은 secure_frame_verify_and_decrypt()가 이 함수 대신 ctr_state_next_outgoing()으로 카운터를 재구성해 복호화만 시도할 뿐, 수신한 카운터가 실제로 유효한 범위인지는 검증하지 않는다.

4.3.3 ctr_state_persist / ctr_state_load

int ctr_state_persist(void);

int ctr_state_load(void);

설명: 현재 카운터 상태를 비휘발성 저장소에 저장/불러온다. ctr_state_next_outgoing()을 호출할 때마다 즉시 기록. ctr_state_load()는 상태 파일이 없어도 실패로 취급하지 않음(-1 반환하되, 처음 필요해지는 시점에 key_store의 초기 CTR로 대체되어 진행함).

기본 파일 경로: "ctr_state.dat" (4.3.4 ctr_state_set_path() 참고).

4.3.4 ctr_state_set_path

```
void ctr_state_set_path(const char *path);
```

매개변수: path [in] 상태 파일 경로.

기본값: "ctr_state.dat" (헤더의 CTR_STATE_DEFAULT_PATH).

설명: ctr_state_load()/ctr_state_persist()가 기본값 대신 사용할 파일 경로 지정. 이 프로젝트의 main.c는 마스터일 땐 "ctr_state.dat", 슬레이브일 땐 "ctr_state_recv.dat"로 나눠 쓴다 (한 프로세스에서 모드를 오갈 때 카운터가 섞이지 않도록).

```
ctr_state_set_path("ctr_state_recv.dat");  
  
ctr_state_load(); // 실패해도 진행 - 첫 사용 시 key_store의 initial_ctr로 대체됨
```

[표 16] ctr_state_set_path 사용 예시

4.3.5 ctr_state_reset

```
void ctr_state_reset(uint8_t slave_addr, key_direction_t dir);
```

매개변수: slave_addr [in], dir [in].

설명: (slave_addr, dir)의 카운터 상태를 지워 처음 상태로 되돌려 다음 호출이 key_store의 initial_ctr부터 다시 시작. 같은 프로세스 안에서 같은 (addr, dir)로 반복 테스트할 때 (예: 셀프 테스트를 여러 번 실행) 이전 실행의 카운터 잔재로 어긋나는 것을 막는 데 사용. 호출 즉시 ctr_state_persist()도 함께 실행되므로 파일에도 리셋 된 상태가 반영된다.

4.4 framing/secure_frame.h

4.4.1 secure_frame_build

```
size_t secure_frame_build(const secure_frame_t *frame, uint8_t *out);
```

매개변수: frame [in], out [out] 바이트 버퍼.

설명: frame을 [addr][ciphertext][hmac] 형태로 out 버퍼에 붙이고 길이를 반환한다.

4.4.2 secure_frame_parse

```
int secure_frame_parse(const uint8_t *in, size_t len, secure_frame_t *frame);
```

매개변수: in [in] 바이트 버퍼, frame[out, secure_frame_parse], len [in].

설명: in의 len바이트를 frame으로 파싱. 성공 시 1, len이 addr+hmac을 담기에도 부족하면 0 반환.

HMAC 검증은 하지 않는 순수 구조 변환으로, 신뢰할 수 없는 입력(네트워크/시리얼에서 막 받은 바이트)에는 이 함수만 단독으로 쓰지 말고 secure_frame_verify_and_decrypt()를 사용해야 한다.

4.4.3 secure_frame_encrypt_and_build

```
int secure_frame_encrypt_and_build(uint8_t slave_addr, const uint8_t *plaintext_pdu, size_t pdu_len, key_direction_t dir, uint8_t *out, size_t *out_len);
```

매개변수: slave_addr [in], plaintext_pdu [in] Function Code+데이터, pdu_len [in] MODBUS PDU 길이, dir [in], out [out] 출력 버퍼, out_len [out].

Pdu_len은 최대 SECURE_FRAME_MAX_PDU(253) 바이트, out은 호출자가 최소 SECURE_FRAME_MAX_WIRE_LEN(288)바이트로 준비해야 함

설명: PDU 앞에 주소, 뒤에 표준 Modbus CRC16을 붙여 ADU로 만들고, (slave_addr, dir) 키로 암호 키 및 카운터를 조회해 송신 카운터로 LEA-CTR 암호화 후 HMAC-LSH256 계산, 완성된 [addr][ciphertext][hmac]을 out에 저장한다. 성공 시 0, key_store_lookup() 실패나 pdu_len이 SECURE_FRAME_MAX_PDU보다 클 시 -1을 반환한다.

동작/경계 조건: 내부적으로 ctr_state_next_outgoing(slave_addr, dir, pdu_len)을 정확히 한번 호출한다. 이 함수를 호출할 때마다 그 (addr, dir)의 카운터가 실제로 전진하므로, 같은 프레임을 다시 만들어서 재전송하면 안 된다(재전송하려면 이미 만들어진 out 바이트를 그대로 다시 보내야 함, 이 함수를 또 호출하면 안 됨).

`uint8_t wire[SECURE_FRAME_MAX_WIRE_LEN];`

```

size_t wire_len;

uint8_t pdu[] = {0x03, 0x00, 0x00, 0x00, 0x02}; /* Read Holding Registers, addr 0, qty 2
*/

if (secure_frame_encrypt_and_build(1, pdu, sizeof(pdu), DIR_MASTER_TO_SLAVE, wire,
&wire_len) != 0) {

    printf("slave 1의 m2s 키가 없음\n");
} else {

    serial_port_write(&sp, wire, wire_len);
}

```

[표 17] secure_frame_encrypt_and_build 사용 예시

4.4.4 secure_frame_verify_and_decrypt

```

secure_frame_status_t secure_frame_verify_and_decrypt(

    const uint8_t *wire, size_t wire_len, uint8_t expected_addr, key_direction_t dir,

    uint8_t *out_addr, size_t *out_ciphertext_len,

    uint8_t *out_pdu, size_t *out_pdu_len,

    uint16_t *out_crc_calc, uint16_t *out_crc_rcv);

```

매개변수: wire/wire_len [in] 수신 바이트, expected_addr [in] 주소 필터, dir [in], out_addr [out] 파싱된 주소, out_ciphertext_len/out_crc_calc/out_crc_rcv [out, 선택] 진단 메시지용 세부값, out_pdu/out_pdu_len [out] 복호화된 순수 PDU

주소 필터 expected_addr에 SECURE_FRAME_ANY_ADDR(0xFF)을 넘기면 필터를 건너뛴.
(1:1 환경 시 응답 테스트용)

out_pdu는 호출자가 최소 SECURE_FRAME_MAX_PDU(253)바이트로 준비

설명: secure_frame_encrypt_and_build()의 역방향. 파싱 → 주소 검사 → 키 조회 → HMAC 검증 → LEA-CTR 복호화 → CRC16 검증 순으로 처리. 유효할 경우

SECURE_FRAME_OK(0)를 반환.

동작/경계 조건:

- HMAC 비교는 memcmp()로 이루어짐. O(N).
- SECURE_FRAME_ERR_CRC는 대개 "HMAC은 맞는데 복호화 결과가 깨짐" = **카운터가 어긋났다는 신호** - 키가 틀렸다면 HMAC 단계에서 먼저 걸리므로, CRC까지 통과 못 했다는 건 인증은 됐지만 잘못된 카운터로 복호화했다는 뜻.
- 내부적으로 ctr_state_next_outgoing()을 호출하므로, 같은 (addr, dir)로 이 함수를 두 번 연달아 부르면 두 번째 호출은 실패.

```
uint8_t addr, pdu[SECURE_FRAME_MAX_PDU];
size_t ciphertext_len, pdu_len;
uint16_t crc_calc, crc_recv;

secure_frame_status_t st = secure_frame_verify_and_decrypt(
    rx_buf, rx_len, SECURE_FRAME_ANY_ADDR, DIR_MASTER_TO_SLAVE,
    &addr, &ciphertext_len, pdu, &pdu_len, &crc_calc, &crc_recv);

switch (st) {
case SECURE_FRAME_OK:
    /* pdu[0..pdu_len) 사용 */
    break;
case SECURE_FRAME_ERR_CRC:
    printf("CRC 불일치 (calc %04X, recv %04X) -- 카운터 어긋남?\n", crc_calc,
        crc_recv);
    break;
default:
    printf("검증 실패: status=%d\n", (int) st);
}
```

[표 18] secure_frame_verify_and_decrypt 사용 예시

반환: 반환하는 secure_frame_status_t 값은 다음 [표 19]에 있다.

값	의미
SECURE_FRAME_OK (0)	성공
SECURE_FRAME_ERR_MALFORMED (-1)	addr+hmac을 담기에도 부족한 길이 - out_addr 등은 채워지지 않음
SECURE_FRAME_ERR_WRONG_ADDR (-2)	expected_addr과 frame.addr이 다름 - out_addr엔 실제 프레임 주소가 채워짐
SECURE_FRAME_ERR_NO_KEY (-3)	key_store에 등록된 키 없음
SECURE_FRAME_ERR_HMAC (-4)	HMAC 불일치 - 잘못된 키이거나 프레임이 변조됨
SECURE_FRAME_ERR_CRC (-5)	복호화 후 CRC16 불일치 - 카운터 어긋남 의심

[표 19] secure_frame_verify_and_decrypt 반환값 (secure_frame_status_t)

4.4.5 secure_frame_crc16

uint16_t secure_frame_crc16(const uint8_t *buf, size_t len);

매개변수: buf/len [in].

설명: 표준 Modbus CRC16(다항식 0xA001, ctr_lo 먼저).

do_self_test()가 secure_frame_encrypt_and_build()를 거치지 않고 직접 테스트 ADU를 구성할 때 필요해서 사용. 특별한 예외/경계 조건 없이 모든 입력에 대해 항상 성공.

4.5 modbus/modbus_pdu.h

Modbus_pdu에서 사용하는 Function code는 다음 [표 20]과 같다.

코드	이름	요청 PDU (Function code 제외)	응답 PDU (Function code 제외)	value_or_ qty 의미
0x01	Read Coils	addr(2)+qty(2)	byte_count(1)+data	읽을 개수
0x02	Read Discrete Inputs	addr(2)+qty(2)	byte_count(1)+data	읽을 개수
0x03	Read Holding Registers	addr(2)+qty(2)	byte_count(1)+data(2× qty)	읽을 개수
0x04	Read Input Registers	addr(2)+qty(2)	byte_count(1)+data(2× qty)	읽을 개수
0x05	Write Single Coil	addr(2)+value(2, 0x0000/0xFF00)	요청과 동일(echo)	0=OFF, 0 아니면 ON
0x06	Write Single Register	addr(2)+value(2)	요청과 동일(echo)	레지스터 값 (0- 65535)
0x0F	Write Multiple Coils	addr(2)+qty(2)+byte_co unt(1)+data	addr(2)+qty(2)	쓸 코일 개 수 (값은 자동 생성)
0x10	Write Multiple Registers	addr(2)+qty(2)+byte_co unt(1)+data(2×qty)	addr(2)+qty(2)	쓸 레지스 터 개수 (값은 자동 생성)

[표 20] 사용 Function Code 정리

데모용 데이터 모델: holding_registers/input_registers/coils/discrete_inputs 각 128개(주소 0~127)에 대하여, 첫 사용 시 선형 패턴으로 초기화(holding[i]=0x1000+i 등)되어 테스트 데이터를 즉시 읽을 수 있도록 설정한다. quantity 상한(MODBUS_MAX_READ

_BITS=2000, MODBUS_MAX_WRITE_BITS=1968, MODBUS_MAX_READ_REGISTERS=125, MODBUS_MAX_WRITE_REGISTERS=123)은 libmodbus-master/src/modbus.h와 동일한 값을 따른다.

Modbus 예외 응답(Function code|0x80 + 예외코드)은 다음 [표 21]과 같다.

예외 코드	이름	발생 조건
0x01	Illegal Function	Function code가 위 8종 밖
0x02	Illegal Data Address	주소(+수량)가 128개 데모 테이블 범위를 벗어남
0x03	Illegal Data Value	요청 길이가 그 Function code의 최소 필드보다 짧음 / quantity가 0이거나 프로토콜 상한 초과 / byte_count가 quantity와 안 맞음 / 코일 값이 0x0000~0xFF00이 아님

[표 21] modbus 예외 코드

4.5.1 modbus_build_request

```
int modbus_build_request(uint8_t func, uint16_t addr, uint16_t value_or_qty,
                        uint8_t *out_pdu, size_t *out_pdu_len);
```

매개변수: func [in] Function code, addr [in] 시작 주소(0~65535, 범위 자체는 형식상 항상 유효), value_or_qty [in], out_pdu [out] 최소 SECURE_FRAME_MAX_PDU(253) 바이트, out_pdu_len [out].

설명: 마스터가 요청 PDU를 만드는 함수. 성공 시 1, 지원하지 않는 Function Code이거나 value_or_qty가 그 함수의 quantity 상한(MODBUS_MAX_ * 값)을 벗어나면 0을 반환하고 out_pdu는 건드리지 않는다.

동작/경계 조건:

- 주소 유효성은 검사하지 않는다. addr이 데모 테이블 범위(0~127)를 넘어가도 이 함수는 그냥 성공(1)해서 PDU를 만든다 - 실제 마스터가 슬레이브의 정확한 레지스터 맵을 미리 알 수 없으므로, 범위를 벗어나면 슬레이브가 modbus_build_response()에서 Illegal Data Address 예외로 정상 응답.
- 0x05(Write Single Coil)는 value_or_qty가 0이면 OFF(0x0000), 0이 아니면 ON(0xFF00)으로 취급한다.
- 0x0F/0x10(다중 쓰기)은 실제로 보낼 값을 지정할 방법 없이(테스트 단순화를 위함) 코일은 인덱스 패리티(0,1,0,1,...), 레지스터는 주소+인덱스로 자동 생성된다. 특정 값을 정확히 쓰고 싶다면 이 함수 대신 0x06(단일 레지스터 쓰기)을 반복 호출하거나 직접 PDU를 조립해야 한다.

```
uint8_t pdu[SECURE_FRAME_MAX_PDU];
size_t pdu_len;

/* Read Holding Registers, 주소 0부터 2개 */
if (modbus_build_request(MODBUS_FUNC_READ_HOLDING_REGISTERS, 0, 2, pdu,
&pdu_len)) {
    /* pdu == {0x03, 0x00, 0x00, 0x00, 0x02}, pdu_len == 5 */
}

/* Write Single Coil, 주소 3을 ON */
modbus_build_request(MODBUS_FUNC_WRITE_SINGLE_COIL, 3, 1, pdu, &pdu_len);
/* pdu == {0x05, 0x00, 0x03, 0xFF, 0x00} */
```

[표 22] modbus_build_request 사용 예시

4.5.2 modbus_build_response

```
int modbus_build_response(const uint8_t *req_pdu, size_t req_len,
                          uint8_t *resp_pdu, size_t *resp_len);
```

매개변수: req_pdu/req_len [in] secure_frame_verify_and_decrypt()가 돌려준 out_pdu/out_pdu_len 그대로, resp_pdu [out], resp_len [out].

Resp_pdu는 최소 SECURE_FRAME_MAX_PDU 바이트 길이만큼 제공해야 함.

설명: 슬레이브가 요청을 해석해 데이터 모델에 적용(쓰기)/조회(읽기)하고 응답 PDU를 만드는 함수. req_len이 0이면 회신 불가로 0 반환(호출자는 회신 생략). 그 외에는 항상 1을 반환하며 resp_pdu에 정상 응답 또는 예외 응답이 담긴다(호출자는 어느 쪽인지 구분할 필요 없이 그대로 암호화해 보내면 됨).

modbus_pdu.c의 실제 검사 순서에 따른 예외 발생 조건은 아래 [표 23]과 같다.

예외	발생 조건 (해당 Function Code에서)
Illegal Data Value (0x03)	req_len이 그 함수의 최소 길이(읽기 5바이트, 단일 쓰기 5바이트, 다중 쓰기는 byte_count 포함 최소 6바이트)보다 짧음
	qty가 0이거나
	MODBUS_MAX_READ_BITS/MODBUS_MAX_WRITE_BITS/MODBUS_MAX_READ_REGISTERS/MODBUS_MAX_WRITE_REGISTERS 상한 초과
	다중 쓰기(0x0F/0x10)에서 요청에 실려온 byte_count가 qty로부터 계산한 기대 값과 다름
Illegal Data Address (0x02)	단일 코일 쓰기(0x05)에서 값이 0x0000도 0xFF00도 아님
	(uint32_t)addr + qty > MODBUS_TABLE_SIZE
Illegal Function (0x01)	레지스터/코일 주소 범위를 넘어가는 경우
	- addr/qty가 각각 uint16_t라 오버 플로우 없이 32비트로 안전하게 더해 비교
Illegal Function (0x01)	Function Code가 위 8종 어디에도 속하지 않음 (default 분기)

[표 23] 예외가 발생하는 정확한 조건

성공 시 응답 형태: 단일 코일/레지스터 쓰기(0x05/0x06) 성공은 요청 5바이트를 그대로 memcpy한 반사(echo) 응답이고, 다중 코일/레지스터 쓰기(0x0F/0x10) 성공은 수신 데이터를 반영하지 않고 func+addr+qty 5바이트만 새로 조립한 응답 (두 경우 모두 표준 Modbus 사양과 일치).

```
uint8_t resp[SECURE_FRAME_MAX_PDU];

size_t resp_len;

/* 위에서 만든 Read Holding Registers 요청(pdu, pdu_len)에 대한 응답 */
if (modbus_build_response(pdu, pdu_len, resp, &resp_len)) {
    /* 정상: resp == {0x03, 0x04, 0x10, 0x00, 0x10, 0x01} (레지스터 0,1의 초기값)
       예외라면 예: resp == {0x83, 0x02} (Illegal Data Address) */
}

/* 범위를 벗어난 주소로 Illegal Data Address 유발 */
{
    uint8_t bad_req[] = {0x03, 0x00, 0x7F, 0x00, 0x02}; /* addr=127, qty=2 ->
127+2=129 > 128 */

    modbus_build_response(bad_req, sizeof(bad_req), resp, &resp_len);

    /* resp == {0x83, 0x02} */
}
```

[표 24] modbus_build_response 사용 예시

4.6 demo/ 공용 유틸리티

함수	매개변수	설명
void exe_relative_path(c onst char *argv0, const char *filename, char *out, size_t out_size)	argv0 [in], filename [in], out [out], out_size [in]	"<argv0 디렉터리>/filename" 문자열 조립. keys.txt/카운터 파일/로그 파일을 실행 파일 위치에 고정하는 데 사용
int demo_load_keys(co nst char *argv0)	argv0 [in, NULL 허용]	keys.txt를 5가지 후보 경로로 순서대로 탐색해 로드 (cwd → 실행파일 옆 → keymgmt/keys.txt → security/keymgmt/keys.txt → ../keymgmt/keys.txt). 불러온 항목 수(>0) 또는 실패 시 <=0 반환
long modbus_t35_us(lon g baud)	baud [in]	Modbus RTU 스펙의 T3.5 유희 간격을 마이크로초 단위로 계산 (실측 기반 20ms 최솟값 포함)
void log_open(const char *path) 외 log_close _atexit/log_detail/l og_summary/print_ hex	—	파일 로깅 유틸리티 (secure_send_demo/secure_recv_demo 전용)

[표 25] demo/ 공용 유틸리티 함수들

기본값/경계 조건: demo_load_keys()는 argv0가 NULL이어도 동작하지만 그 경우 "실행파일 옆" 경로 후보는 건너뛴다(나머지 4개 후보만 시도). 5개 후보를 모두 실패하면 0 이하를 반환하며 key_store는 변경되지 않는다.

modbus_t35_us()는 계산 값이 20ms(20000us) 미만이면 20ms로 올림 한다. (실측 기반)

log_open()을 호출하지 않고 log_detail/log_summary를 호출하면(파일이 안 열려 있으면) 해당 로그는 버려진다. main.c는 파일 로그를 쓰지 않으므로 이 유틸리티들을 아예 쓰지 않고 자체 printf 기반 출력만 사용한다.

```
/* secure_recv_demo.c의 실제 초기화 순서 (main.c는 demo_log 대신 printf만 사용) */
char log_path[512], ctr_path[512];

exe_relative_path(argv[0], "recv_log.txt", log_path, sizeof(log_path));
log_open(log_path);
atexit(log_close_atexit);

if (demo_load_keys(argv[0]) <= 0) {
    log_summary("Could not load keys.txt\\n");
    return 1;
}

exe_relative_path(argv[0], "ctr_state_recv.dat", ctr_path, sizeof(ctr_path));
ctr_state_set_path(ctr_path);
ctr_state_load();

serial_port_open(&sp, argv[1], baud, (int) modbus_t35_us(baud));
```

[표 26] 유틸리티 함수 사용 예시

5. 데모 프로그램 사용법

5.1 secure_demo (main.c)

```
choi@addon2:~/security $ ./secure_demo
-----
[master] mode, keys: not loaded, port: (none), slave addr: 1
1. 마스터/슬레이브 모드 전환
2. 키 초기화
3. 단위 테스트
4. 환경 설정
5. 실행 (보안 통신 1회 수행)
6. 종료
-----
```

[그림 2] secure_demo 실행화면

- **1. 모드 전환:** is_master 플래그를 뒤집어서 master / slave 모드로 설정. 기본값은 1(Master).
- **2. 키 초기화:** demo_load_keys()로 keys.txt 로드.
- **3. 단위 테스트:** 포트에 실제로 연결하지 않고 알고리즘 동작만 테스트. **6.1** 참고.
- **4. 환경 설정:** 포트(빈 값=현재 유지, -를 입력 시 포트 없음/파일 사용) → baud → slave 주소(마스터일 땐 보낼 대상, 슬레이브일 땐 자신의 주소) → Function Code(1/2/3/4/5/6/15/16 중 하나) → 시작 주소 → value_or_qty(Function Code에 따라).
- **5. 실행:** 마스터면 Function Code로 요청을 만들어 암호화 후 전송(포트 미설정 시 sent_frame.bin에 기록). 슬레이브면 수신 프레임을 검증/복호화하고, 실제 포트가 있으면 응답까지 암호화해 회신(포트 없이 파일로 읽은 경우는 검증/복호화만 진행).
- ① 터미널 두 개에서 프로그램 실행
- ② 하나의 프로세스는 master로, 하나의 프로세스는 slave로 설정
- ③ key 파일을 불러옴
- ④ master측에서 환경 설정으로 port, baud rate, slave addr, Function code 선택한다. slave측도 환경 설정을 통해 port, baud rate, slave addr 입력 및 설정한다.
- ⑤ slave측에서 응답 대기하도록 5번을 입력하고, master에서 5번을 입력한 후 전송 및 응답 결과를 확인한다.

5.2 secure_send_demo / secure_rcv_demo

secure_send_demo [port] [baud] [count] # 포트 생략 시 sent_frame.bin에 기록

secure_rcv_demo port [baud] [count] # 포트 필수

secure_send_demo는 함수 0x10(Write Multiple Registers) 프레임을 레지스터 1개~123개까지 크기를 늘려가며 count로 설정한 횟수만큼 전송해 모든 크기에서 프레임이 정상 생성되는 지 테스트한다(다른 Function Code는 다루지 않음). secure_rcv_demo는 실제 포트에서 프레임을 받아 검증/복호화하고 응답까지 회신한다.

6. 테스트 방법

6.1 단위 테스트 (secure_demo 옵션 3, do_self_test())

하드웨어 없이 한 프로세스 안에서 공개 API를 직접 호출해 검증하는 세 개의 체크로 구성되어 있다.

- **encrypt-path 체크** (테스트 주소 0xF0, DIR_MASTER_TO_SLAVE):
secure_frame_encrypt_and_build()를 호출한 뒤, 알고 있는 ctr_low=0으로 직접 HMAC/복호화/CRC를 검증.
- **decrypt-path 체크** (테스트 주소 0xF0, DIR_SLAVE_TO_MASTER): ctr_low=0으로 직접 만든 프레임을 secure_frame_verify_and_decrypt()에 넘겨 검증.
- **Modbus_build_response() 체크**: MODBUS 요청에 따라 정상 및 예외 응답을 제대로 생성하는 지 검증 (modbus_pdu_self_test())

실행: secure_demo 실행 후 3 입력.

기대 출력:

```
Running crypto logic self-test (no serial port involved)...
PASS (encrypt path): secure_frame_encrypt_and_build -> parse -> HMAC verify ->
decrypt -> CRC check OK (5 byte PDU, 41 byte wire frame)
PASS (decrypt path): hand-built frame -> secure_frame_verify_and_decrypt OK (5 byte
PDU, 41 byte wire frame)
[PASS] 0x01 Read Coils success
[PASS] 0x02 Read Discrete Inputs success
[PASS] 0x03 Read Holding Registers success
```

```

[PASS] 0x04 Read Input Registers success
[PASS] boundary: addr+qty == table size (128) succeeds
[PASS] boundary: addr+qty == table size+1 (129) -> Illegal Data Address
[PASS] boundary: qty == protocol max (125) succeeds
[PASS] boundary: qty == protocol max+1 (126) -> Illegal Data Value
[PASS] boundary: req_len < 5 -> Illegal Data Value
[PASS] exception: qty=0 -> Illegal Data Value
[PASS] exception: addr+qty > table size -> Illegal Data Address
[PASS] exception: unsupported function code -> Illegal Function
[PASS] boundary: Write Single Coil invalid value -> Illegal Data Value
[PASS] boundary: Write Multiple Coils byte_count mismatch -> Illegal Data Value
[PASS] boundary: Write Multiple Registers byte_count mismatch -> Illegal Data Value
[PASS] boundary: Write Multiple Coils addr+qty overflow -> Illegal Data Address
[PASS] 0x05 Write Single Coil success
[PASS] 0x05 Write Single Coil actually persisted (read-back)
[PASS] 0x06 Write Single Register success
[PASS] 0x06 Write Single Register actually persisted (read-back)
[PASS] 0x0F Write Multiple Coils success
[PASS] 0x0F Write Multiple Coils actually persisted (read-back)
[PASS] 0x10 Write Multiple Registers success
[PASS] 0x10 Write Multiple Registers actually persisted (read-back)
24 passed, 0 failed (24 total)
Self-test: ALL PASS

```

[표 27] 셀프 테스트 정상 시 기대 출력

같은 프로세스 안에서 여러 번 실행해도(옵션 3을 반복 선택) 매번 PASS해야 정상이다.

6.2 파일 왕복 테스트 (하드웨어가 없을 경우)

포트를 설정하지 않은 상태에서:

1. 마스터로 2(키 초기화) → 4(환경 설정, 원하는 Function Code/주소/수량 입력)
→ 5(실행) — sent_frame.bin에 씀.
2. 새 프로세스(또는 1로 모드 전환)에서 2 → 5(실행) — sent_frame.bin을 읽어 검증/

복호화. Recovered PDU 확인.

한계: 포트가 없으면(sp == NULL) 슬레이브가 응답을 만들지 않으므로, 이 방법은 요청 방향(DIR_MASTER_TO_SLAVE)의 PDU 인코딩만 검증한다. 응답 생성(modbus_build_response())은 앞서 언급한 **6.1 단위 테스트**로 검증하면 된다.

6.3 실제 장비(Pi) 검증 절차

SSH 연결이 되어있는 CM5 기기에서 검증하려면 다음 [표 29]와 같이 git에서 소스를 복사해 빌드한다.

```
ssh <pi-host>  
git clone https://github.com/siwoncool0530/modbus_security.git ~/security  
cd ~/security  
git checkout main
```

[표 29] git clone

keys.txt는 .gitignore 대상이라 clone에 안 들어오므로 필요시 별도로 옮겨야 한다:

개발 머신에서

```
scp keymgmt/keys.txt <pi-host>:~/security/keymgmt/keys.txt
```

[표 30] keys.txt Pi로 복사

빌드 (Pi는 aarch64라 Makefile이 자동으로 NEON 백엔드 선택):

```
make                # secure_send_demo, secure_rcv_demo, secure_demo 모두 빌드  
  
# NEON 관련 문제 있으면: make clean && make NO_NEON=1
```

[표 31] make를 이용하여 빌드

실행:

```
./secure_demo        # 대화형 (마스터/슬레이브, 셀프테스트 등)  
  
./secure_rcv_demo /dev/ttyAMA0 115200
```

[표 32] 실행

이후 업데이트 필요 시:

```
cd ~/security  
git pull origin main  
make clean && make
```

[표 33] 코드 업데이트

- git remote -v로 origin이 실제로 설정됐는 지 먼저 확인해야 한다 (clone했다면 자동으로 설정됨).
- keys.txt, ctr_state*.dat 둘 다 .gitignore 대상이라 git pull로 덮어써지지 않는다. 카운터 및 로컬 키 편집 시에도 안전하다.
- 로컬에 커밋 안 된 변경사항이 있으면 git pull이 충돌할 수 있으니, 그 전에 git status로 확인 권장.

6.4 체크리스트

- 단위 테스트(옵션 3) 여러 번 연속 실행 시 매번 ALL PASS 여부
- 슬레이브 주소를 프레임과 다르게 설정하면 Frame addressed to slave X, not us (configured as Y) -- ignoring로 무시됨
- 4.5의 8개 Function Code 모두 환경 설정 → 실행으로 요청 PDU가 잘 생성되는지
- Pi에서 위 항목들을 다시 실행해도 동일하게 PASS

7. 참조 문헌

- [1] modbus.org, "MODBUS over Serial Line Specification and Implementation Guide V1.02," 2006.
- [2] modbus.org, "MODBUS APPLICATION PROTOCOL SPECIFICATION V1.1b3," 2012.
- [3] 국가보안기술연구소, "128비트 블록암호 LEA 규격서," 2013.
- [4] D. Hong, J.-K. Lee, D.-C. Kim, D. Kwon, K. H. Ryu and D.-G. Lee, "LEA: A 128-Bit Block Cipher for Fast Encryption on Common," in *international workshop on information security applications*, 2013.
- [5] D.-C. Kim, D. Hong, J.-k. Lee, W.-H. Kim and D. Kwon, "LSH: A New Fast Secure Hash Function Family," in *International Conference on Information Security and Cryptology*, 2014.
- [6] 국가보안기술연구소, "해시함수 LSH 규격서," 2014.
- [7] 국가보안기술연구소, "블록암호 LEA 소스코드 사용 매뉴얼(v1.0)," 2015.
- [8] 국가보안기술연구소, "LSH 소스코드 사용 매뉴얼(v1.0)," 2016.